SAP Variant Dynamic Date — ABAP Fetch Guide

A reference for correctly resolving SAP selection variant dynamic date variables in ABAP and using them to fetch the right data.

Table of Contents

1. The Problem
2. How Dynamic Date Variables Work
3. The Two Storage Layers
4. The Correct ABAP Code
5. Inspecting a Variant First
6. What the Resolved Output Looks Like
7. Common Dynamic Variable Names
8. When You Are Inside the Report Itself

The Problem

When an SAP selection variant stores a dynamic date variable (e.g., "Today", "Current Month", "First Day of Month"), the raw value stored in the variant is **not a resolved date**. It is a reference token that must be resolved at runtime.

If you read the variant with the wrong function module, you get the unresolved token string back — and your `SELECT` either silently returns wrong data or errors out entirely.

How Dynamic Date Variables Work

```
Variant stored on database
  └── BUDAT: LOW = 'TODAY'          ← raw token, not a date

RS_VARIANT_CONTENTS_GET called at runtime (e.g. 2026-03-17)
  └── BUDAT: LOW = '20260317'       ← resolved to actual date
```

The resolution happens inside `RS_VARIANT_CONTENTS_GET` at the exact moment you call it. The result always reflects "now" — so calling it tomorrow gives tomorrow's date for `TODAY`, this

month's range for `CURRENT_MONTH`, and so on.

The Two Storage Layers

Function Module	What It Returns	Use It?
`RS_VARIANT_CONTENTS`	Raw parameters — dynamic tokens may be unresolved	❌ Not for dynamic dates
`RS_VARIANT_CONTENTS_GET`	Fully resolved parameters — dynamic dates converted to actual values	✅ Always use this

> **Rule of thumb:** If the variant could contain any dynamic date variable, always use `RS_VARIANT_CONTENTS_GET`. Using `RS_VARIANT_CONTENTS` alone will return unresolved tokens like `TODAY` instead of an actual date.

The Correct ABAP Code

```abap
```

```
REPORT z_variant_dynamic_date.

DATA: lv_report   TYPE rsvar-report  VALUE 'YOUR_REPORT_NAME',
      lv_variant  TYPE rsvar-variant VALUE 'YOUR_VARIANT_NAME',
      lt_seltab   TYPE STANDARD TABLE OF rsparams,
      ls_sel      TYPE rsparams,
      lr_date     TYPE RANGE OF d,
      ls_date     LIKE LINE OF lr_date.

"-- Step 1: Fetch and RESOLVE the variant (dynamic dates resolved here) --
CALL FUNCTION 'RS_VARIANT_CONTENTS_GET'
  EXPORTING
    i_report  = lv_report
    i_variant = lv_variant
  TABLES
    i_seltab  = lt_seltab   " output: resolved params/select-options
  EXCEPTIONS
    not_found = 1
    OTHERS    = 2.

IF sy-subrc <> 0.
  MESSAGE 'Variant not found or error' TYPE 'E'.
ENDIF.

"-- Step 2: Transfer the resolved date range into a usable RANGES table --
LOOP AT lt_seltab INTO ls_sel
  WHERE selname = 'DATE_FIELD'.   " <-- your actual selname from the variant

  ls_date-sign   = ls_sel-sign.
  ls_date-option = ls_sel-option.
  ls_date-low    = ls_sel-low.
  ls_date-high   = ls_sel-high.
  APPEND ls_date TO lr_date.
  CLEAR ls_date.
ENDLOOP.

"-- Step 3: Use the range in your SELECT --
SELECT *
  FROM your_table
  INTO TABLE @DATA(lt_result)
  WHERE posting_date IN @lr_date.
```

Key points

- `lt_seltab` (type `STANDARD TABLE OF rsparams`) receives all resolved selection fields from the variant.
- The `LOOP ... WHERE selname = 'DATE_FIELD'` filters to just the field(s) you care about. Replace `DATE_FIELD` with the actual `selname` from your variant (e.g., `BUDAT`, `ERDAT`, `VBELN_DATE`).
- The `sign`, `option`, `low`, `high` fields map directly to a RANGES table — no conversion needed.
- The resolved range works in any `IN @lr_date` clause exactly like a manually defined range.

Inspecting a Variant First

Before writing the fetch code, you need to know what `selname` values the variant contains. Run this snippet in SE38 or a quick test program:

```abap
DATA lt_seltab TYPE STANDARD TABLE OF rsparams.

CALL FUNCTION 'RS_VARIANT_CONTENTS_GET'
  EXPORTING
    i_report  = 'YOUR_REPORT_NAME'
    i_variant = 'YOUR_VARIANT_NAME'
  TABLES
    i_seltab  = lt_seltab.

" Option A: break and inspect in debugger
BREAK-POINT.

" Option B: display inline
cl_demo_output=>display( lt_seltab ).
```

This shows you:

- All `selname` values present in the variant (e.g., `BUDAT`, `ERDAT`)
- Whether each field is a parameter (`P`) or a select-option (`S`) via the `KIND` field
- The actual resolved `LOW` and `HIGH` values at the moment of the call

---

What the Resolved Output Looks Like

For a variant with dynamic date variable "Today" on field `BUDAT`, after `RS_VARIANT_CONTENTS_GET` runs on 2026-03-17:

SELNAME	KIND	SIGN	OPTION	LOW	HIGH
`BUDAT`	`S`	`I`	`BT`	`20260317`	`20260317`

For "Current Month" on the same date:

SELNAME	KIND	SIGN	OPTION	LOW	HIGH
`BUDAT`	`S`	`I`	`BT`	`20260301`	`20260331`

The dynamic variable is fully resolved — your `WHERE date IN lr_date` receives actual date values, not tokens.

Common Dynamic Variable Names

These are the internal tokens SAP stores in variants when a user selects a dynamic date option:

User-facing label in variant	Internal token stored
Today	`TODAY`
Current week (Mon–Sun)	`CURRENT_WEEK`
Previous week	`LAST_WEEK`
Current month	`CURRENT_MONTH`
Previous month	`LAST_MONTH`
Current year	`CURRENT_YEAR`
Previous year	`LAST_YEAR`
First day of current month	`FIRST_DAY_OF_MONTH`
Last day of current month	`LAST_DAY_OF_MONTH`

`RS_VARIANT_CONTENTS_GET` resolves all of these automatically at call time. You never need to resolve them manually.

When You Are Inside the Report Itself

If your code runs **as part of the report** (i.e., the variant was already applied to populate the selection screen), the dynamic dates are already resolved in your `SELECT-OPTIONS` fields. In that case, just use them directly in your `SELECT`:

```abap
" s_budat is already resolved from the variant at selection screen time
SELECT *
  FROM bkpf
  INTO TABLE @DATA(lt_docs)
  WHERE budat IN @s_budat.
```

`RS_VARIANT_CONTENTS_GET` is only needed when you are **programmatically driving another report's variant from outside it** — for example, in a batch job, a wrapper report, or a custom scheduler that applies variants to reports before submitting them.

Flow Summary

```
Variant name (user-supplied or hardcoded)
        |
        ▼
RS_VARIANT_CONTENTS_GET
  └── Resolves dynamic tokens → actual date ranges
        |
        ▼
lt_seltab (STANDARD TABLE OF rsparams)
  └── Loop, filter by selname → populate lr_date (RANGE OF d)
        |
        ▼
SELECT ... WHERE date IN @lr_date
        |
        ▼
Correct result set
```

Last updated: 2026-03-17